



MERCANTEC DATAHOUSE
Vejleder: Mathias Gaarsdal Steenberg

Lavet af:
Teis Binder & Phamishan Packiaseelan

Indholdsfortegnelse

Indholdsfortegnelse	1
Kravspecifikation	2
Indledning	2
Mål	2
Funktionelle krav	2
Ikke-funktionelle krav	2
Teknologivalg	3
Frontend (Phamishan)	3
Backend (Teis)	3
Vejledning (Frontend, Phamishan)	3
Køre flutter frontend	3
Register	4
Login	5
Hjemmeskærm	8
Grupper	9
Votes	10
Challenges	12
Venner	13
Profil	17
Vejledning (Backend, Teis)	18
Auth service Endpoints	19
API service Endpoints	21
Implementering af Frontend (Phamishan)	26
Flutter	26
Filstruktur	27
Implementering af Backend (Teis)	29
Begrænsninger	34
Tidspres:	34
Hjemmeside:	34
Sikkerhed:	34
Api'er:	34

Kravspecifikation

Indledning

ChallengeCircle er en app for vennegrupper, hvor man kan konkurrere mod hinanden i udfordringer man selv opretter. Når man logger ind på appen, så har man muligheden for at oprette en gruppe, hvor man kan invitere ens venner og bagefter oprette udfordringer, som giver x-antal point ved fuldendelse. Man skal derfor også kunne se stillingen mellem sig selv og sine andre venner.

Mål

- Udvikle en app der samler venner, som gerne vil konkurrere mod hinanden.
- Skabe et sted for vennegrupper et sted, hvor de kan motivere hinanden.

Funktionelle krav

- Man skal som bruger kunne logge ind og ud af appen.
- Man skal som bruger kunne deltage i en gruppe / flere grupper.
- Man skal som bruger kunne fuldføre en udfordring.
- Man skal som bruger kunne se et leaderboard over gruppen.
- Man skal som gruppe admin kunne tilføje medlemmer.
- Man skal som gruppe admin kunne fjerne medlemmer.
- Man skal som gruppe admin kunne godkende når en bruger indsender en fuldført udfordring.

Ikke-funktionelle krav

- Login skal være sikker.
- Hurtigt respons.

Teknologivalg

Vi har valgt en række forskellige teknologier, som hjælper os med at udvikle vores projekt bedst muligt og dermed også valgt et par teknologier som vi også har lidt erfaringer med fra forrige projekter.

Frontend (Phamishan)

- Flutter

Flutter er et oplagt valg til frontend, nu hvor vi gerne vil have en mobil-app, da det både kan køres på IOS og Android fra den samme kode. Vi har også muligheden for at udgive som en Desktop applikation og en hjemmeside senere hen (med samme kodebase), hvis nødvendigt.

Backend (Teis)

Vores backend består af en Nodejs app med Express.js til API service. Der bliver brugt MySQL til vores database som kører i sin egen docker container. Vores API'er bruger PRISMA ORM til at beskrive alle vores queries og kommunikere med databasen.

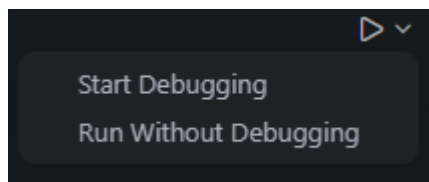
Vores docker containers er på en debian 12 server som er hosted hos Hetzner. Denne server tillader kommunikation via tcp igennem port 4000 og 4001 til henholdsvis vores auth service og api servicen.

Serveren kan kontaktes på: <http://157.180.66.30/>

Vejledning (Frontend, Phamishan)

Køre flutter frontend

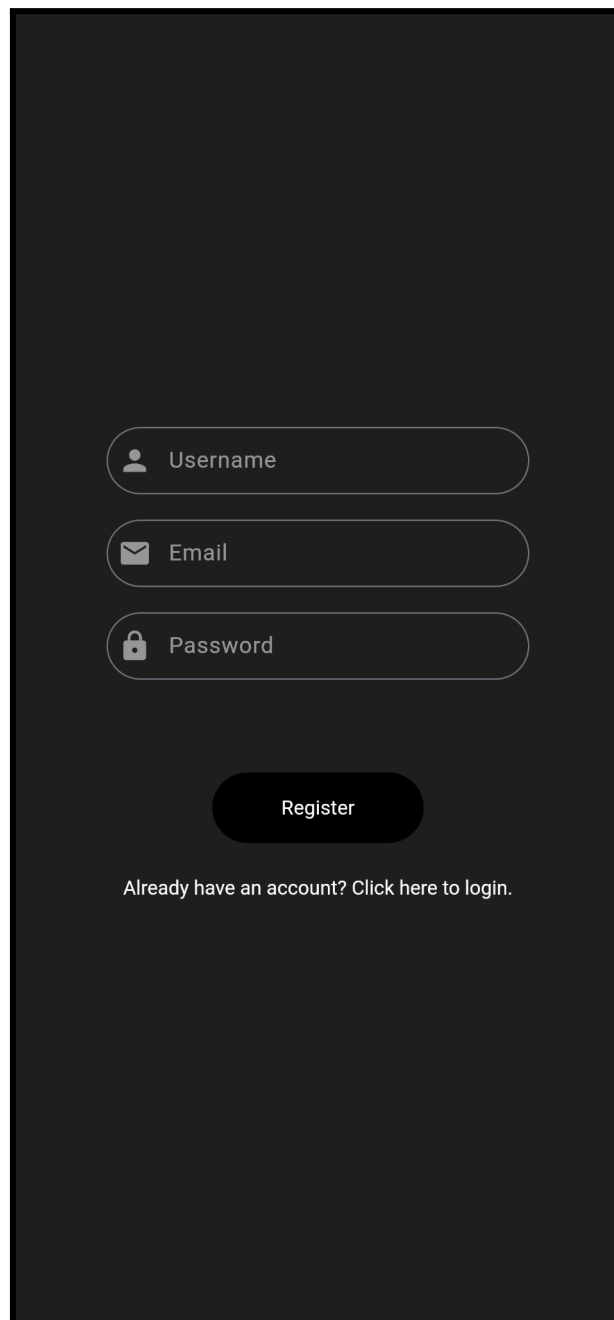
Hvis man gerne vil køre flutter frontend lokalt er det vigtigt at man har en flutter installeret. Derefter skal man i terminalen, CD ind til "cc_app", skrive "pub get" for at hente de flutter biblioteker vi bruger og kan derefter skrive "flutter run" eller klikke ind på main.dart og klikke på "Run without Debugging".



Derefter vil den spørge om en evt. emulator, men i vores tilfælde er en browser fin, for eksempel, Google Chrome.

Google OAuth vil ikke virke da man ikke har vores .env eller firebase_options.dart.

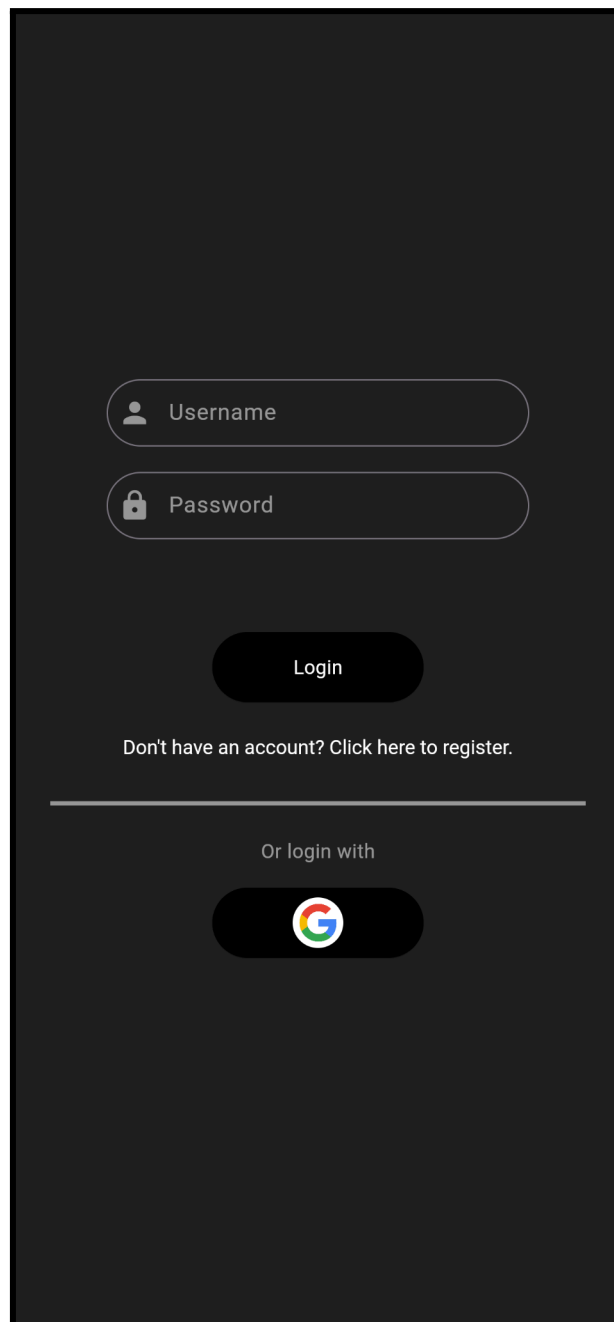
Register

A screenshot of a registration form on a dark background. The form consists of three vertically stacked input fields, each with a light gray border and rounded corners. The first field has a person icon and the label 'Username'. The second field has an envelope icon and the label 'Email'. The third field has a lock icon and the label 'Password'. Below these fields is a dark, rounded rectangular button with the text 'Register' in white. At the bottom of the form, there is a line of text: 'Already have an account? Click here to login.'

Screenshot af reigster siden

Register-siden er en simpel side bygget op på tre input-felter, en register-knap og en knap, der fører hen til login, hvis man allerede har et login. Dermed har brugeren en mulighed for at gå tilbage til login-siden, hvis han/hun har trykket ind på register-siden ved en fejl. Når input-felterne er udfyldt, så vil brugeren blive skiftet over til login siden, hvor han/hun kan logge ind med den nye bruger, de lige har lavet.

Login



Username

Password

Login

Don't have an account? [Click here to register.](#)

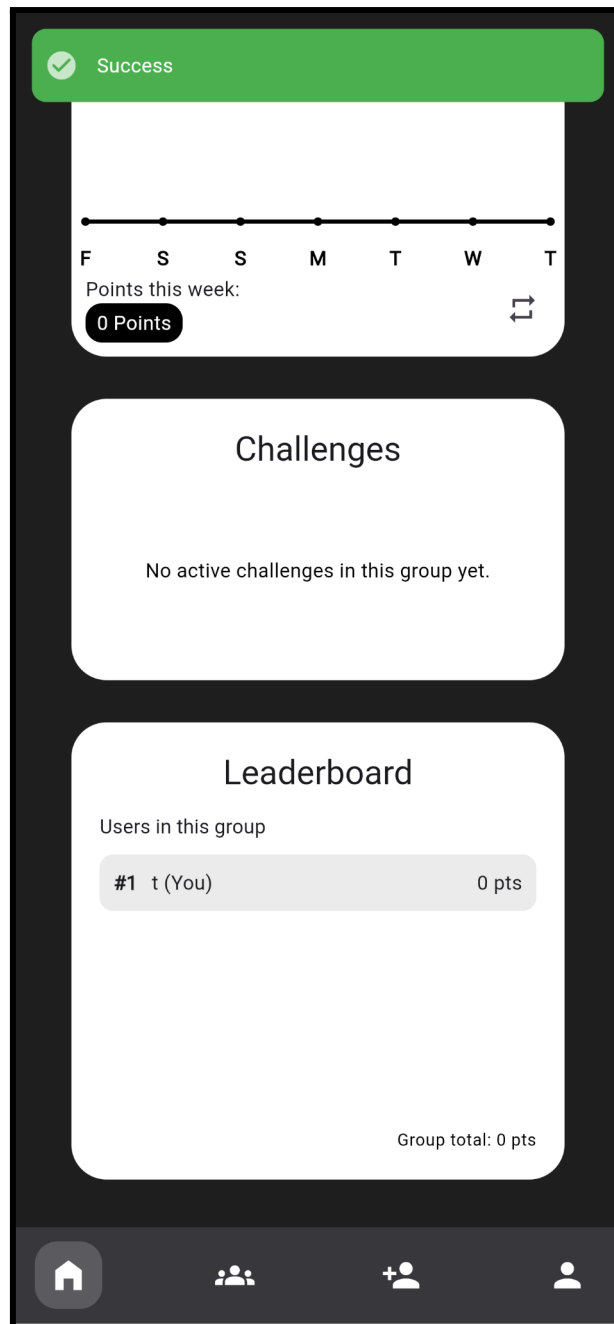
Or login with



Screenshot af login siden

Login-siden er bygget op på to input-felter, en login-knap, en knap til register-siden og en knap, hvor man kan logge ind med google.

Hvis man ikke har et login, så er der to muligheder, enten at registrere sig ved hjælp af vores registreringsformular eller man kan vælge at logge ind med Google og dermed bruge ens google oplysninger som information. Efter man er logget ind bliver man skiftet over til hjemmeskærmen, medmindre der kommer en fejl ved login. Alt efter om man de rette oplysninger eller ikke, så vil der komme en "toast", hvilket er en lille popup i toppen af skærmen, som informere brugeren, hvad der er sket.



Screenshot af home siden (med success toast efter login)

! Please fill out all required fields.

Username

Password

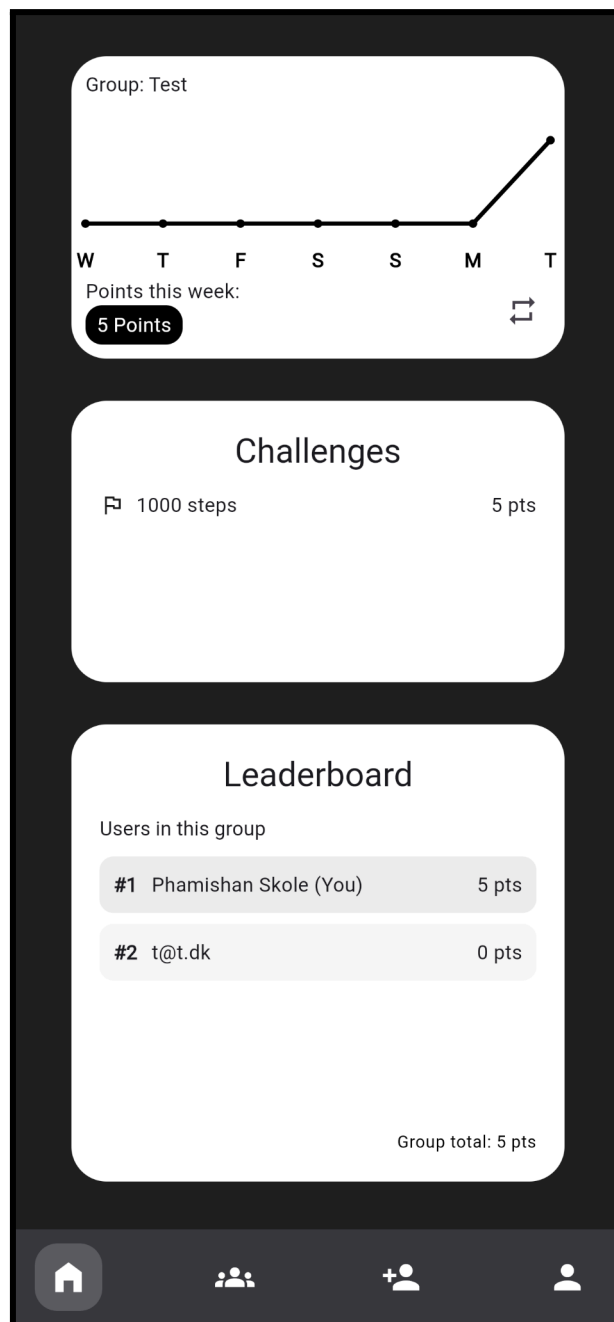
Login

Don't have an account? [Click here to register.](#)

Or login with

Screenshot af login siden (med failed toast)

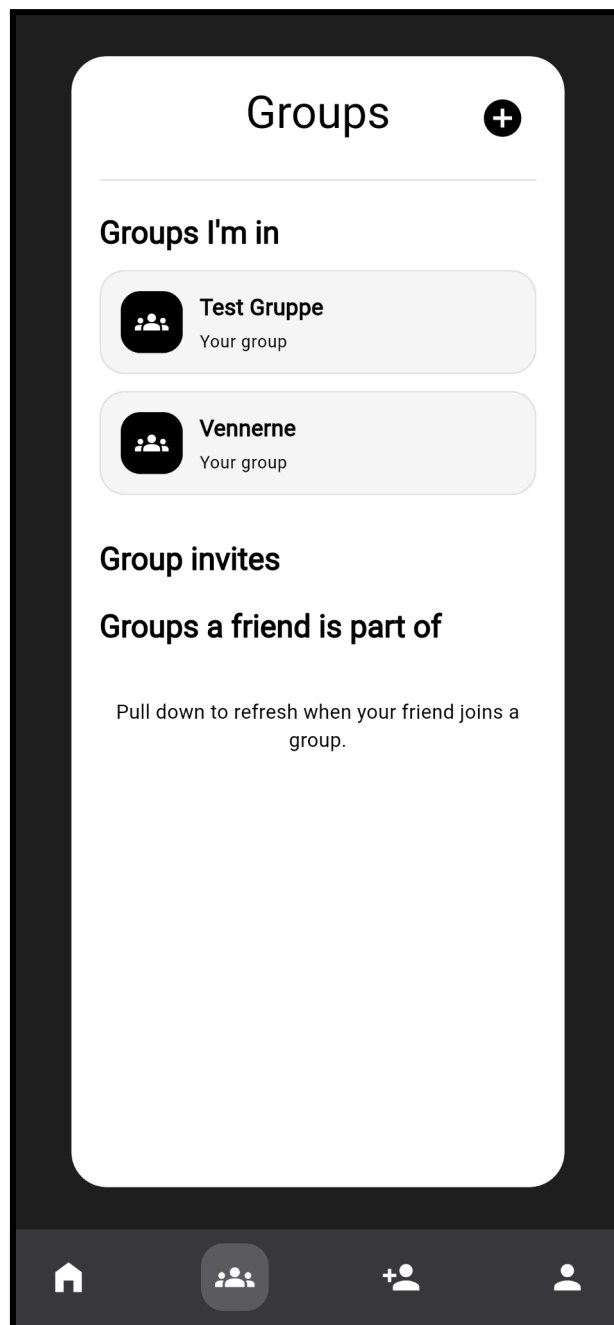
Hjemmeskærm



Screenshot af home siden

Når man er logget ind, bliver man ført ind til hjemmeskærmen. Her er der tre “widgets”, som viser informationer fra den gruppe, man har valgt. Widget nr. 1 er en graf over, hvor mange point man har optjent i løbet af de sidste 7 dage, i den gruppe man har valgt. Nr. 2 er en widget til at vise de challenges der er i en gruppe og nr. 3 er et widget af leaderboardet over de personer som er med i gruppen. Alle tre widget kigger på den gruppe man har valgt i widget nr. 1.

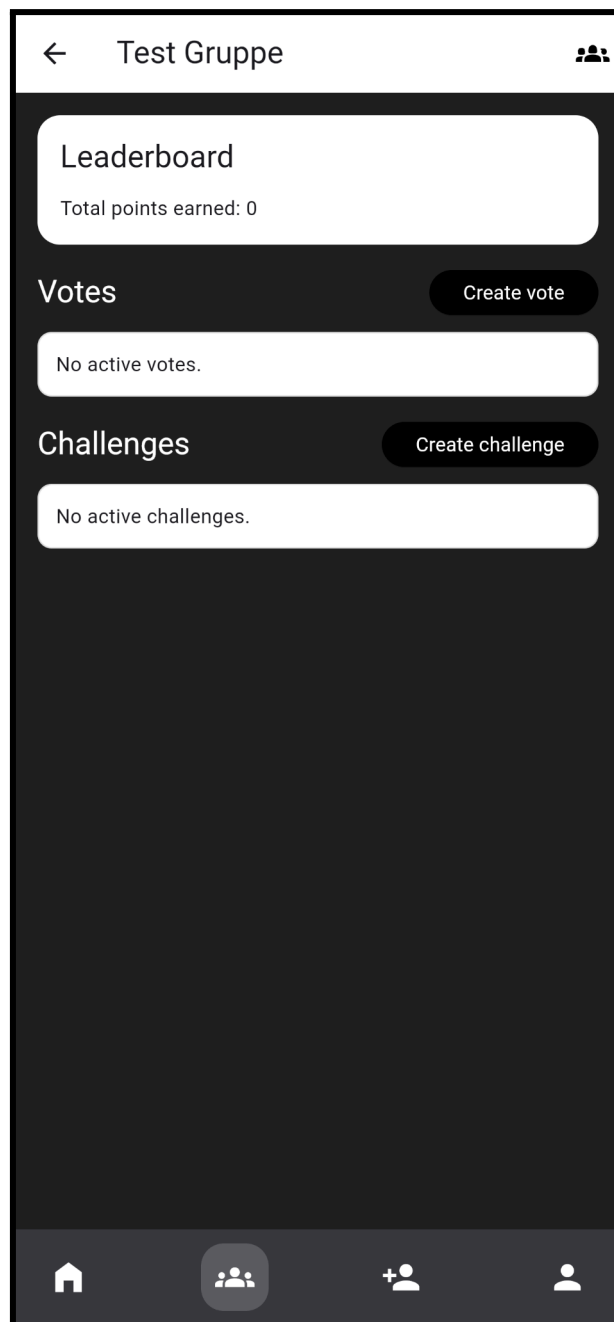
Grupper



Screenshot af groups siden

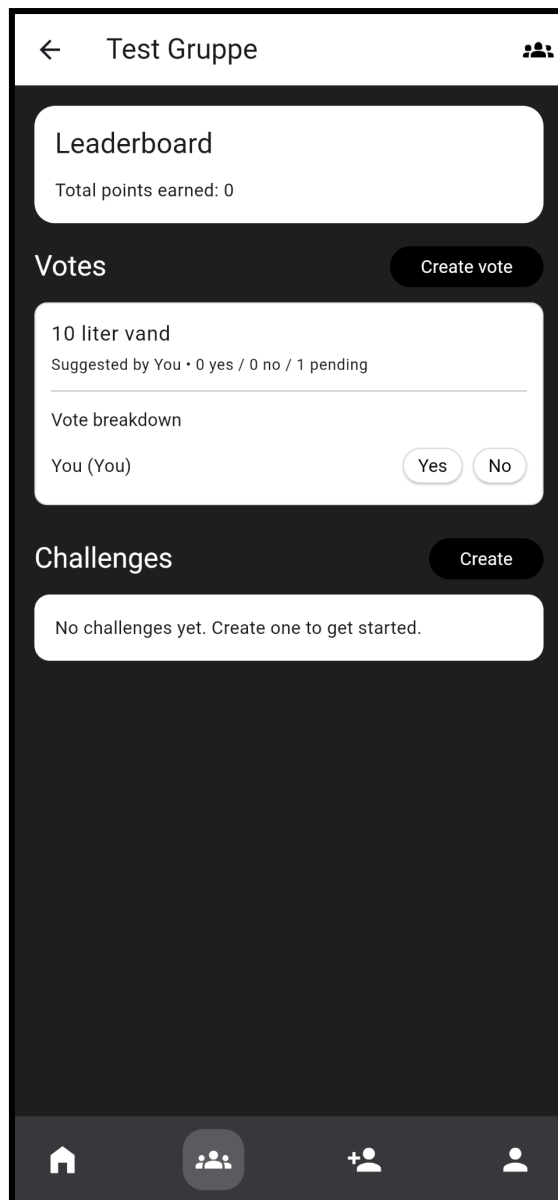
Når man er inde på groups siden, så vil der være ingen grupper at se, man skal derfor oprette en gruppe, oppe i højre hjørne, hvor man efter tryk, vil komme ind på en side, med et input-felt til gruppenavn. Efter man har indtastet et gruppenavn og trykket på “Create group”, så vil man, alt efter man har oprettet succesfuldt eller ej blive ført tilbage til groups siden og der vil komme en toast der vil bekræfte at en gruppe er blevet oprettet succesfuldt, hvis ikke, så vil der komme en toast, som

Votes



Screenshot af group-detail siden

Når man går ind på en gruppe, så bliver ført ind til group-detail, det er en template, som bliver brugt til alle gruppe så det er essentielt kun navnet og gruppe id, som bliver ændret. Det resulterer også i at votes og challenges bliver holdt gruppe vist, da de bliver hentet fra gruppe id'et.

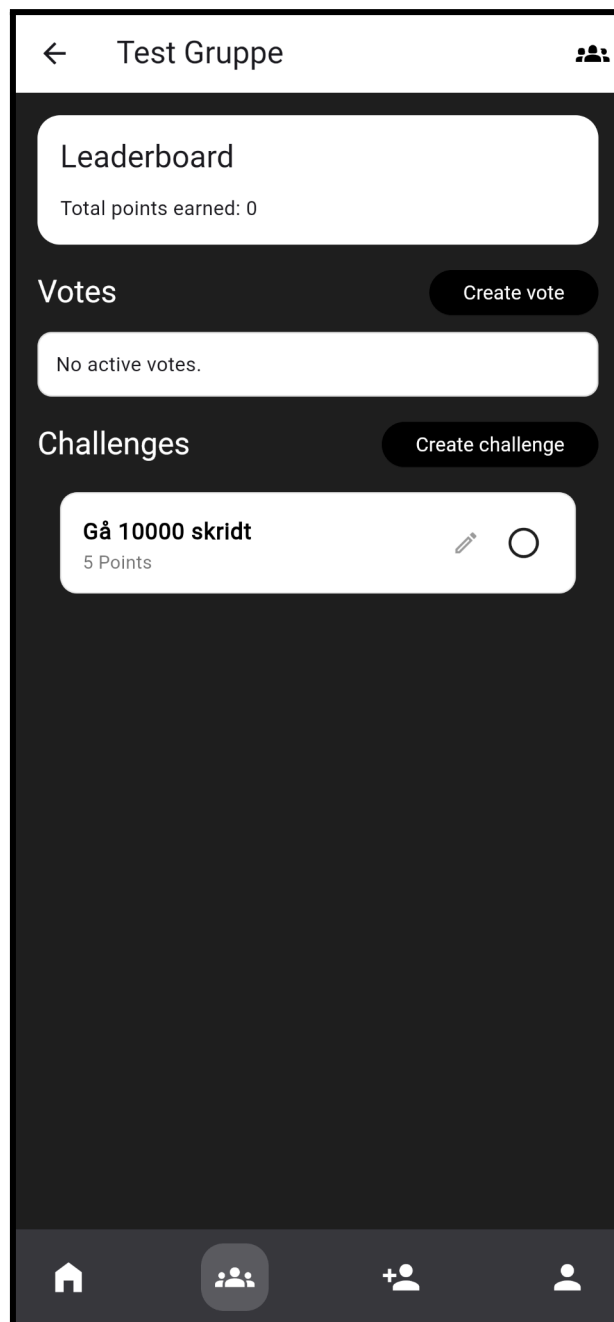


Screenshot af group-detail siden, hvor der er lavet en vote

Votes er lavet med tanken om at man gerne vil have muligheden for at oprette afstemninger for evt. nye challenges man gerne vil indføre. Man skriver sit forslag og derefter vil der være en vote hos alle brugere inde på gruppen, hvor de kan tage stilling til forslaget.

Dette er et gammelt screenshot, hvor det var implementeret til at virke med local storage, men da vi ikke har haft tid til at implementere det fuldendt virker det ikke på den "fulde" version.

Challenges

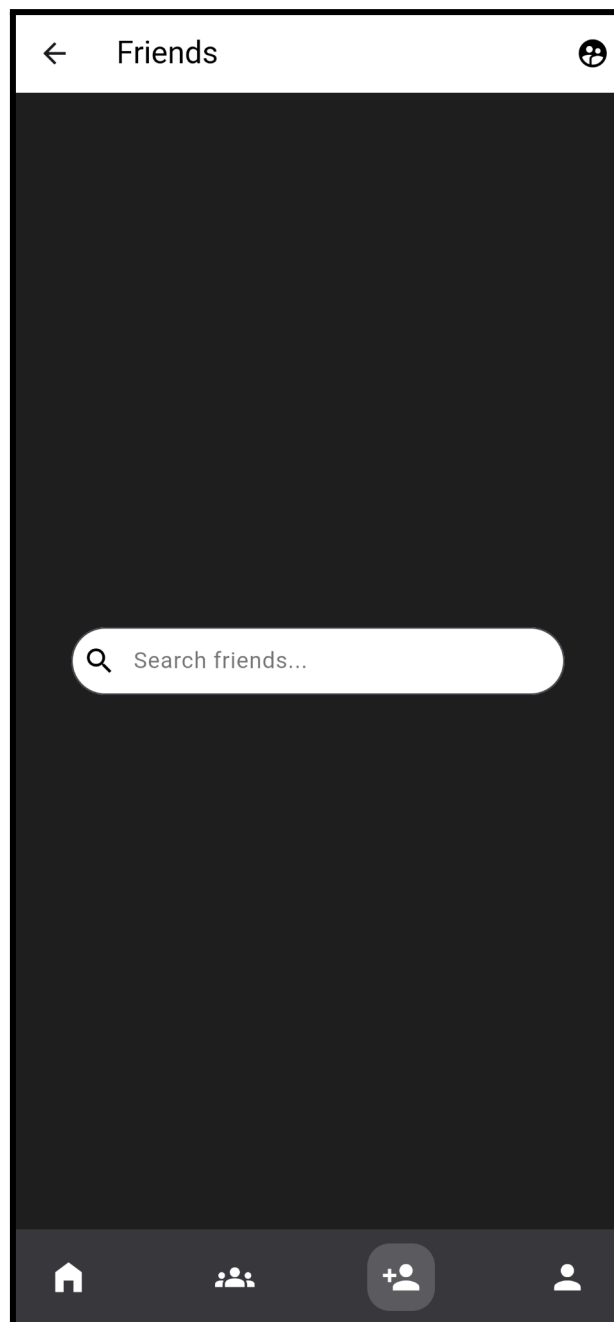


Screenshot af group-detail siden

Challenges er her, hvor der bliver oprettet udfordringer til alle i gruppen. Det kunne for eksempel være, "Gå 10000 skridt" og give 5 point ved udførsel.

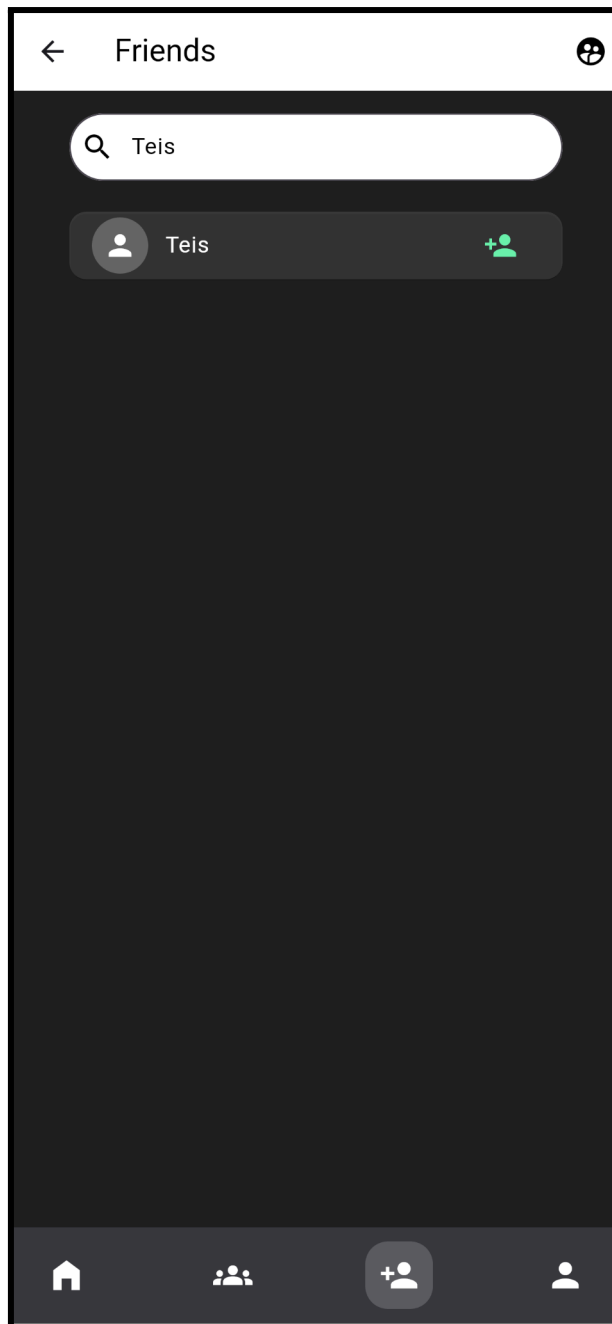
Dette er også en funktion, vi ikke har kunnet gøre færdigt, da vi ikke har haft tid til at implementere det fuldstændigt. Man kan oprette en challenge, hvor den også bliver oprettet i backenden, men når man har trykket på checkboxen, så popper der en toast frem, men uden svar til backenden.

Venner



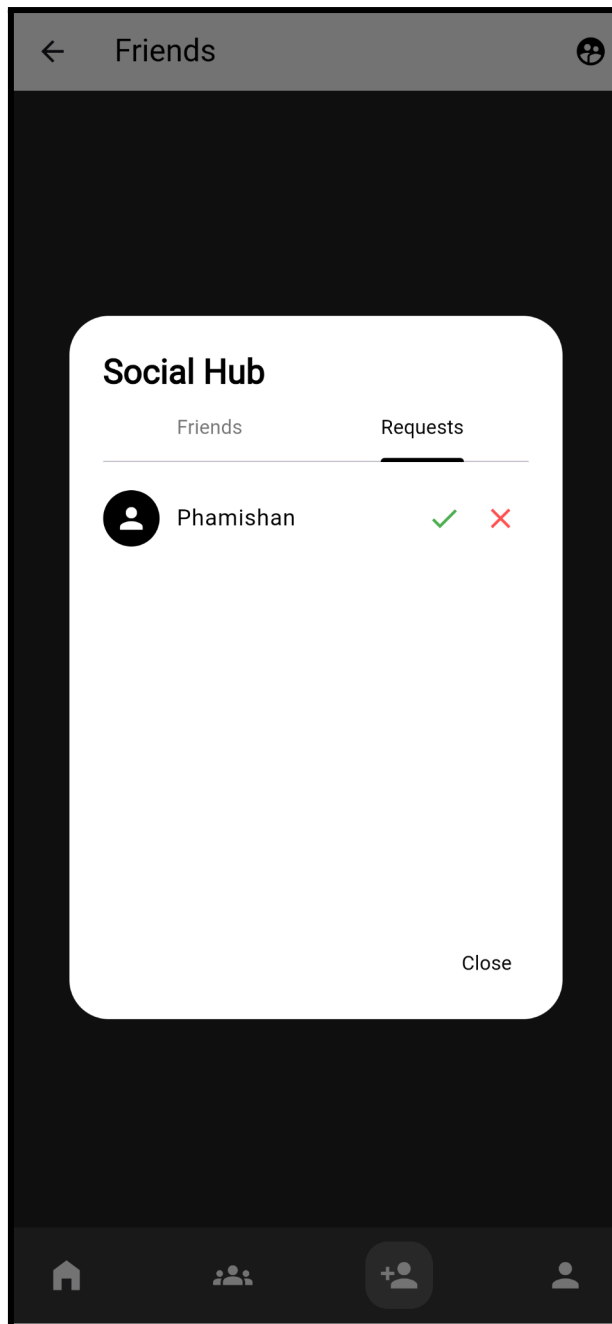
Screenshot af friends siden

Venner siden er her, hvor man kan søge efter brugere. Midt på siden er der et input-felt, som efter indtastning søger gennem brugere på databasen og finder dem frem. Oppe i højre hjørne er der en knap til at vise ens venner, som henter ens venneliste ved tryk og viser friend requests.



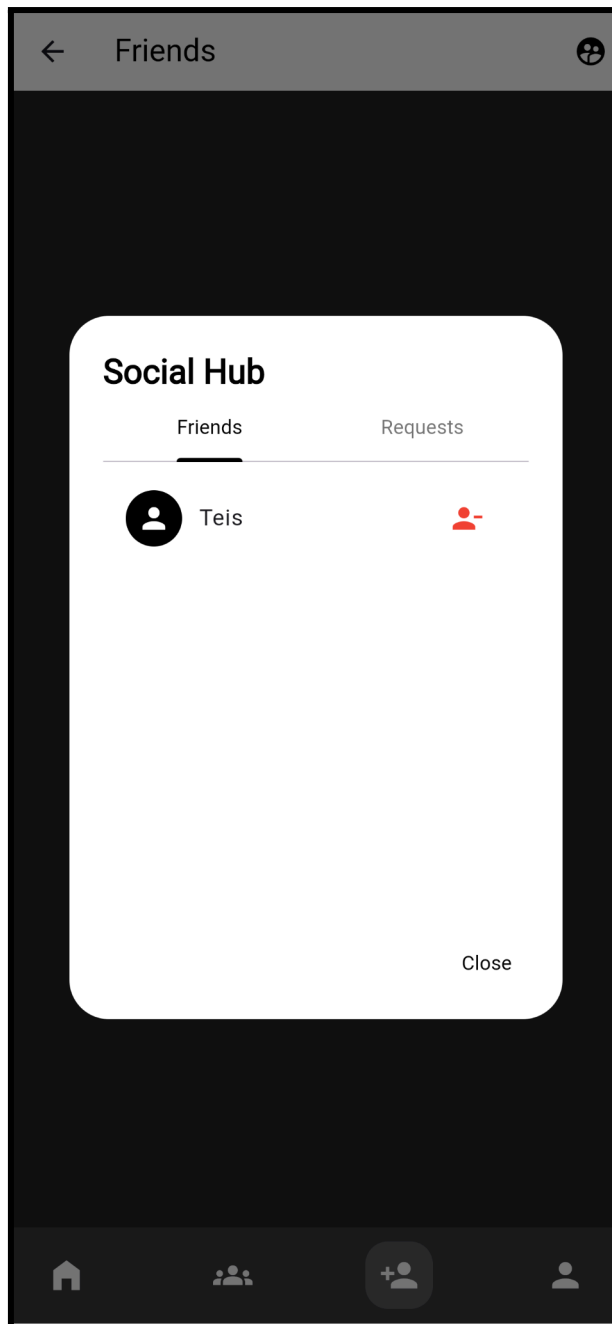
Screenshot af friends siden, hvor man søger efter en bruger

Når man søger efter en bruger, så vil brugeren komme frem med et add-user ikon. Ved tryk vil den så sende en venneanmodning til den anden bruger, som så kan gå ind på sin bruger og se anmodningen.



Screenshot af friends siden, hvor man har fået en request

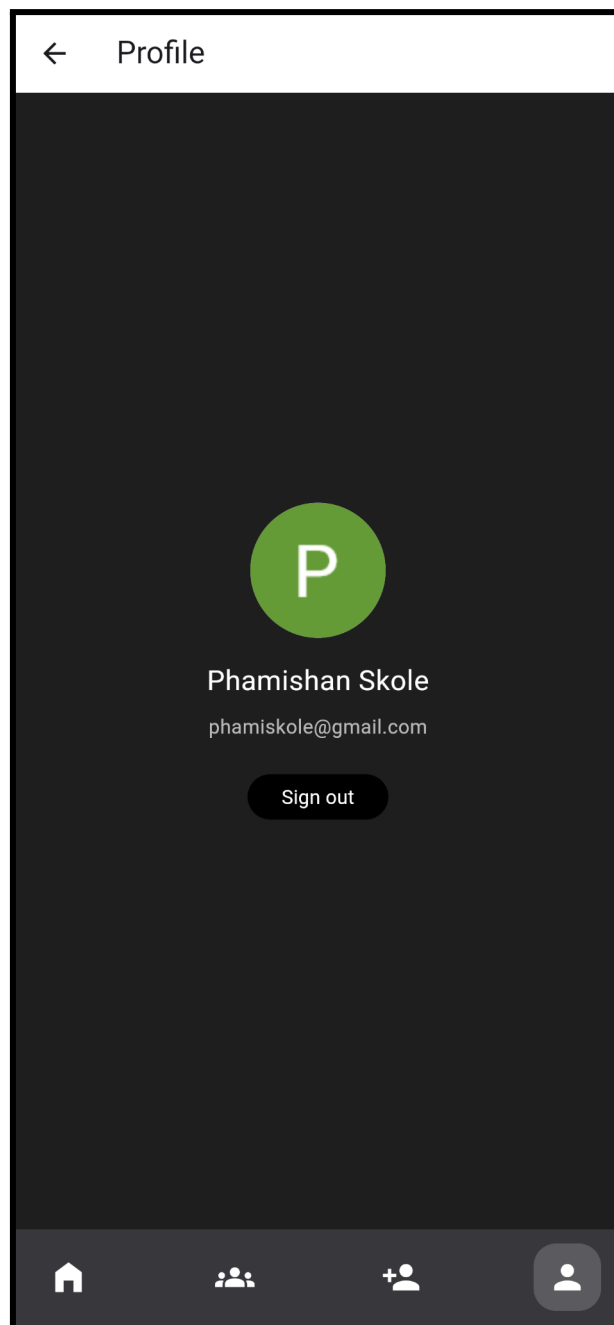
Når en bruger går ind på friends siden og trykker på venner-ikonet oppe i højre hjørne så vil der komme et popup med to tabs, designet til at vise ens venner og venneanmodninger. Når man går ind på venneanmodninger og trykker på det grønne check-ikon, så vil det blive sendt til backenden, og man er nu venner.



Screenshot af friends siden, hvor man kan se vennelisten

Den bruger som sendte anmodningen, vil nu kunne se den anden bruger på ens venneliste. Man kan også trykke på remove-user-ikonet, hvor den så vil sende anmodningen til backend og fjerne brugeren som ven.

Profil



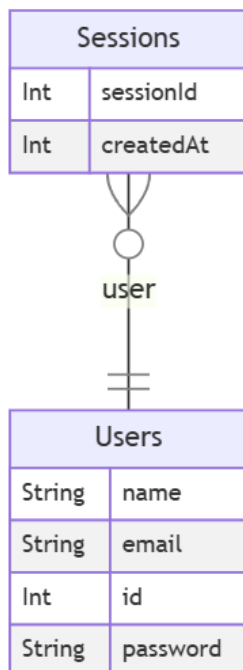
Screenshot af profile siden

Inde på profil siden kan man se ens information. Hvis man er logget ind med Google OAuth, så vil ens google navn, email og profilbillede blive vist. Under ens oplysninger er der en logud knap, som logger en ud og fører en ud til login skærmen.

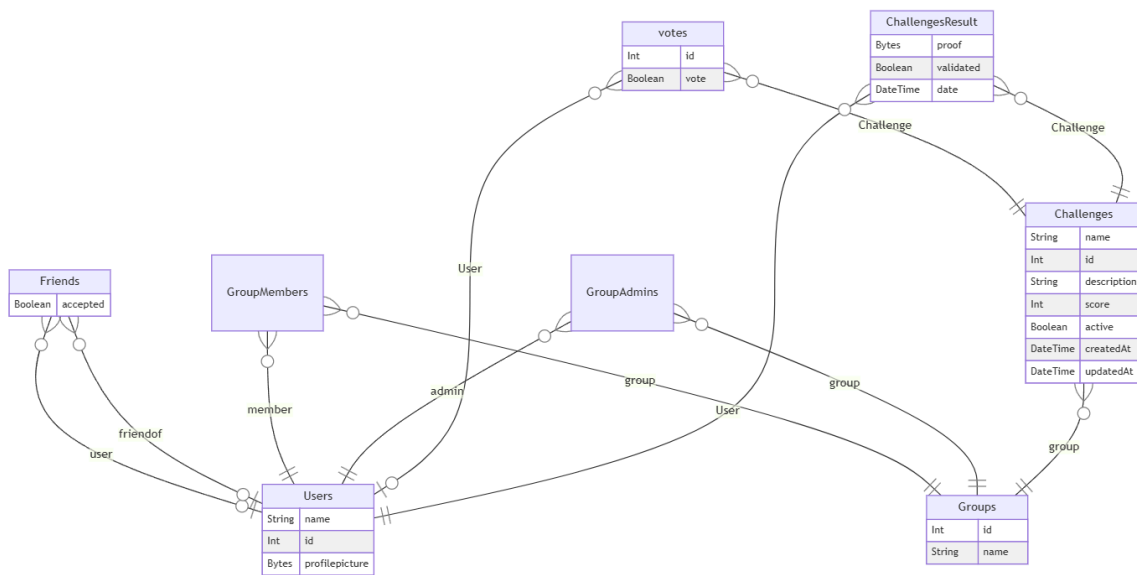
Vejledning (Backend, Teis)

For at sætte backend up skal man bruge docker. Når koden er hentet fra github skal man være i root folder og køre 'docker compose up --build' Det vil bygge de tre containere og oprette de relevante tabeller i databasen som er den officielle mysql container hentet fra dockerhub. Derefter kan api'erne kontaktes på de externe porte 4000 og 4001.

AUTH service ER Diagram



CHALLENGE service ER Diagram



Auth service Endpoints

/Login

method: POST

request: { username: string, password: string }

response:

status: 200 | 400

{ message: "Success", token: Auth token | No user found | Incorrect password | Please fill out all required fields }

/logout

Method: POST

header: authorization: bearer 'auth token'

Possible responses:

status: 200 | 403

{ success | unauthorized }

/createUser

Method: POST

request:

body: { username: string, email: string, password: string }

response:

status: 200 | 400 | 500

{ Success | Please fill out all required fields | User already exists | User failed to create | An error occurred elsewhere while creating the user }

/deleteUser

Method: DELETE

header: authorization: bearer 'auth token'

Possible responses:

200 | 403

{ success | unauthorized }

/API/verify

method: POST

request: header: authorization: 'bearer' authtoken

response:

200 | 403

{ Authorized | Unauthorized }

/API/getPublicKey

Method: GET

response:

200

{ RSA public key }

API service Endpoints

Alle endpoints udstedet i API servicen kræver authorization header med token.

/edit/:id/user

Method: PUT

Request:

body: {username: string, profilepicture: blob, email: string, password: string}

Response:

200 | 500

{ Success | Error from Auth | Internal Error}

/auth/delete/:id/user

Method: DELETE

Request:

Header: Authorization: 'bearer' auth token

Response:

200 | 500

{ Success | error }

/user/findusers/:query

Method: GET

response:

200 | 500

{

 name: string;

 id: number;

 profilepicture: Uint8Array<ArrayBuffer>

} | { internal error }

/user/getUser/:id

Method: GET

response:

200 | 400

{name: string

id: number

profilepicture: Uint8Array<ArrayBuffer>

} | {No user found}

/user/addFriend

Method: POST

request:

{

 userId: Int

 friendId: Int

}

response:
status: 200 | 400
{ Friend request sent } | { Request already pending }, { Unable to send friend request }

/user/acceptFriend

Method: PUT
request: { userId: int, friendId: int }
response:
Status: 200
{ Friend request accepted }

/user/removeFriend

Method: DELETE
request:
{
 userId: String
 friendId
}

Response:
status: 200 | 500
{ Friend removed | Error message }

/user/getFriendRequests

Method: POST
request:
{
 id: Int
}

Response:
status: 200 | 400
{ list of friend requests | Error message }

/user/getFriends

Method: POST
request: { id: string }
Response:
status: 200 | 400
{ list of friends | Error message }

/user/:id/getgroups

Method: GET
response:
200
{ list of groups | Not a part of any groups }

/group/create

Method: POST

request: { userId: string, groupName: string }

response:

status: 200 | 400 | 500

{ Group created successfully } | { Please fill out all required fields } | { An error occurred | error message }

/group/:groupId/getMembers

Method: GET

response:

status: 200

```
{
  "id": Int,
  "name": String,
  "members": [
    {
      "member": {
        "name": String,
        "id": Int
      }
    },
    {
      "member": {
        "name": String,
        "id": Int
      }
    }
  ]
}
```

/group/:groupId/addUser

Method: POST

Request:

{ adminId: Int, userId: Int }

Response:

200 | 400 | 403

{ Successfully added user } | { Please fill out all required fields } | { Unauthorized }

/group/:groupId/addAdmin

Method: POST

Request:

{ adminId: Int, userId: Int }

Response
200 | 400 | 403
{ success } | { Please fill out all required fields } | { Unauthorized }

/group/:groupId/kickUser

Method: POST
Request: { adminId: Int, userId: Int }

Response
status:
200 | 403
{ User removed successfully } | { unauthorized }

/group/:groupId/delete

Method: Post
Request: { adminId: Int }
Response
status:
200 | 400 | 403
{ Group successfully deleted } | { invalid request body } | { unauthorized }

/group/:groupId/edit

Method: PUT
Request: { userId: Int, name: string }

Response
status:
200 | 403
{ Success } | { Unauthorized }

/challenge/create

Method: POST
Request: { challengeName: String, score: Int, description: string, active: boolean, groupId }

Response
status:
200 | 400
{ Challenge created } | { Please fill out all required fields }

/challenges/:groupId/getAll

Method: POST
Response:
status: 200
{
 "id": Int,
 "name": String,
 "challenges": [
 {

```

        "name": String,
        "id": Int,
        "description": String,
        "score": Int ,
        "groupId": Int,
        "active": Boolean,
        "createdAt": "2026-06-12T10:43:23.824Z",
        "updatedAt": "2026-06-12T10:43:23.824Z"
      }
    ]
  }
}

```

/challenge/:challengeId/edit

Method: PUT

request:

```

{
  score: int,
  challengeName: String,
  description: String,
}

```

Response:

status: 200 | 400

{ Success } | { Invalid request body }

/challenge/:challengeId/end

Method: PUT

request:

```

{
  GroupId: Int,
  userId: Int,
}

```

Response:

status: 200 | 403

{ Success } | { Unauthorized }

/challenge/:challengeId/delete

method: DELETE

request: {

```

  groupID: Int,
  userId: Int,
}

```

Response: {

200 | 403

{ Challenge deleted } | { Unauthorized }

/challenge/:challengeId/submit

Method: POST

request: {
 userId: Int
 image: Blob
}

Response:

Status:

200 | 500

{ Success } | { An error occurred while submitting the result. }

/challenge/:groupId/scoreboard

method: GET

response:

status: 200

{
 username: score,
 username: score,
 username: score,
}

/challenge/:challengeId/acceptsubmission

Method: Post

request:

{
 adminId: Int,
 userId: Int,
}

response:

status:

200 | 403

{ Submission accepted } | { Unauthorized }

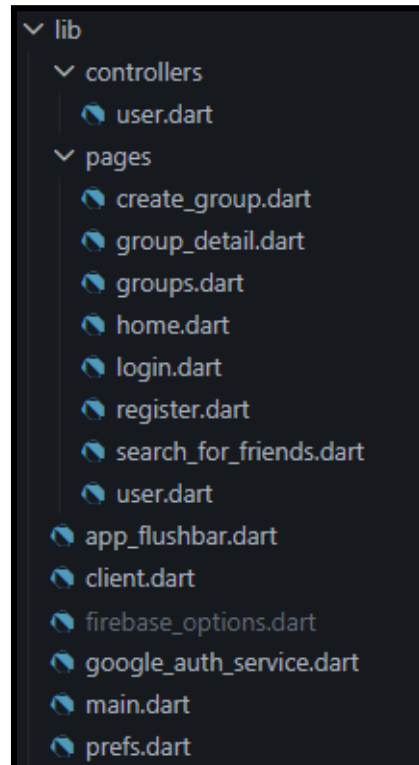
Implementering af Frontend (Phamishan)

Flutter

ChallengeCircle's frontend er udviklet i Flutter og dermed også lavet til flere platforme bare ved hjælp af den ene kodebase. Flutter giver også en mulighed for at bruge biblioteker, som hjælper med at spare tid. Vi har blandt andet brugt "fl_chart" til

at lave vores graf på hjemmeskærmen. Her bliver der brugt vores point data, som så bliver vist på hjemmeskærmen.

Filstruktur



Screenshot af filstrukturen i flutter koden

Vores filstruktur er delt op så vi har et par mapper med funktionalitet som hører sammen og et par enkelte filer, som arbejder alene.

Mappe 1 - Controllers:

I controllers mappen har vi vores user.dart, som er en user controller. Den holder styr på, at vi gemmer vores token, som vi får med fra backenden.

Mappe 2 - Pages:

I pages har vi vores sider. Det er altså de sider som brugeren ser og interagere med. Når vi har dem i en mappe, holder vi et godt overblik over vores sider og gør det også lettere for os at implementere nye ting, hvis nødvendigt.

Så har vi et par enkelte filer som bliver brugt.

Fil 1 - app_flushbar.dart:

App flushbar er et bibliotek, som hjælper med at lave pæne toasts, som vi bruger i vores app til både succesfulde API kald og fejlede API kald. De giver brugeren feedback, så han/hun ved, hvad der sker.

Fil 2 - client.dart:

I client.dart udfører vi alle vores API kald til backenden. Det er her, hvor vi sender og får svar tilbage.

Fil 3 - firebase_options.dart:

Firestore options er vores firebase som holder styr på vores Google OAuth. Den sørger for at vi er forbundet til vores firebase og kan bruge de funktioner vi har tilladt. Bemærk, denne fil er også gråt ud, hvilket betyder at den ikke kommer med op, når vi pusher koden op til GitHub, da den indeholder vores connection til vores firebase.

Fil 4 - google_auth_service.dart:

Google auth service er her, hvor vi har vores Google OAuth logik. Vi bruger et bibliotek ved navn: "google_sign_in_all_platforms" som sørger for at vi kan logge ind med Google.

Fil 5 - main.dart:

Main er hoved filen i flutter. Det er her vores ens app starter og derfor bliver vores main også brugt til at load en gemt token. Hvis der ikke er en token, bliver man ført videre til LoginPage, hvor man så kan logge ind og få en token.

Fil 6 - prefs.dart:

Prefs eller preferences er logikken til at gemme vores token i user controllers. Her bruger vi "shared_preferences" som hjælper med at gemme det rigtigt.

Implementering af Backend (Teis)

Backend er dockeriseret ved brug af en docker compose fil samt to dockerfiles til at sætte miljøet op med de relevante pakker.

```
>Run Service
database:
  image: mysql ## From https://hub.docker.com/\_/mysql/
  restart: unless-stopped
  env_file:
    - db.env
  volumes:
    - ./db_data:/var/lib/mysql
  networks:
    - backend
  ports:
    - 3306:3306
  healthcheck:
    test: mysqladmin ping -h 127.0.0.1 -u $$MYSQL_USER --password=$$MYSQL_PASSWORD
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 10s
```

På billedet vises konfigurationen til mysql database.

Image feltet beskriver basis containeren som bliver hentet fra docker hub.

Der defineres en volume som relaterer til et sted på enheden den bliver kørt på. Det er for at have persistent data.

Den forbindes til netværket Backend og bliver bundet til intern og ekstern port på containeren. Så er det beskrevet et health check som checker om databasen er oppe at køre.

```

api:
  build: ./backend/api
  restart: unless-stopped
  depends_on:
    database:
      condition: service_healthy
  ports:
    - 4001:3050
  env_file:
    - api.env
  develop:
    watch:
      - action: rebuild
        path: ./backend/api/
        target: /api
        initial_sync: true
        ignore:
          - node_modules/
      - action: rebuild
        path: package.json
  networks:
    - backend
    - frontend

```

Billedet viser configurationen af Api

Vores API og Auth service bygges selv ud fra backend/api og /auth. Begge containerer kræver at databasen er oppe at køre før de selv starter. De har hver sin env fil til configuration af database link og ip til den anden service. Der exposes en ekstern port som i dette scenarie er 4001 som mappes til 3050 internt i containeren.

De er så også forbundet til backendnetværket samt et frontend netværk der dog ikke bliver brugt.

Der er også beskrevet en watch funktion som rebuild containeren når ændringer bliver lavet.

Billedet nedenfor viser et eksempel på en Dockerfile der bliver brugt til at bygge billedet til Auth servicen.

```

1 FROM node:24-slim
2
3 RUN apt-get update -y && apt-get install -y openssl && apt-get install -y wget
4
5 WORKDIR /backend/auth
6
7 COPY package.json package-lock.json ./
8
9 RUN npm ci
10
11
12 COPY . .
13
14
15 CMD ["sh", "-c", "npm run db:deploy && npm run dev"]

```

Dockerfile til auth servicen.

Backend strukturen består af 2 apps. Den ene er i backend/auth.

Den håndtere alt relateret til brugeroprettelse og aktive sessioner.

Der bliver brugt prisma til at kommunikere med databasen og til håndtering af tabeller.

Database modellen er beskrevet i schema.prisma som ligger i prisma folderen.

```

model Users {
  name      String
  email      String    @unique
  id         Int        @id @unique @default(autoincrement())
  password   String
  sessions   Sessions[]
}

model Sessions {
  userId     Int
  sessionId  Int        @id
  createdAt  Int
  user       Users @relation(fields: [userId], references: [id])
}

```

@Unique betyder at feltet skal være unikt så data ikke kan gentages.

@id går det felt til den unikke identifier til tabellen.

Sessions [] laver en one to many relation til tabellen session. Det kan også ses på user feltet i Sessions tabellen der har en @relation som relaterer userId feltet til id i user tabellen.

Der bliver duplikeret data imellem de tabeller. feltet navn og id går igen i begge tabeller.

I src mappen har vi den egentlige app fil.

app'en bruger node som runtime og bruger express frameworket til at udstede routerne.

```

var dynamicCorsOptions = function (req: any, callback: any) {
  var corsOptions;
  if (req.path.startsWith('/auth/')) {
    corsOptions = {
      origin: `${process.env.AUTH_URL}`,
    };
  } else {
    corsOptions = {
      origin: '*',
      credentials: true,
    };
  }
  callback(null, corsOptions);
};

```

Billedet ovenover er en implementation af dynamisk cors hvor der kun tillades auth at på ruter som starter med /auth

Den anden er i backend/api. Den håndterer alt andet logik der relaterer til appen. Denne app bruger også prisma ligesom Auth servicen og databasen blive håndteret på samme måde.

```

model Users {
  name      String
  id         Int           @id @unique
  profilepicture Bytes?
  challengesResults ChallengesResult[]
  admin      GroupAdmins[]
  votes      votes[]
  friends     Friends[]    @relation(name: "user")
  friendof    Friends[]    @relation(name: "friendof")
  groups      GroupMembers[]
}

model Friends {
  accepted Boolean @default(false)
  user      Users? @relation(name: "user", fields: [userId], references: [id], onDelete: Cascade)
  friendof Users? @relation(name: "friendof", fields: [friendid], references: [id], onDelete: Cascade)
  userId    Int
  friendid  Int

  @@id([userId, friendid])
}

```

Prisma schemaet i api er meget mere kompliceret.

Der er flere mange til mange relationer og compound id'er

I Friends tabellen som relaterer til user er der et "onDelete: cascade" hvilket gør at hvis en user bliver deletet bliver alle forbundede elementer også fjernet. Det vil sige at brugeren som bliver deletet ikke længere vil vises som forbundet til en anden bruger som "friend"

@@id er et compound id som gør at alle de inkluderede felter fungere som identifies til tabellen.

```

async function auth(req: any, res: any, next: any) {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(400).json('Please log in');
  }

  const token = authHeader.split(' ')[1];

  if (!token) {
    return res.status(400).json('Please log in');
  }
  try {
    const response = await fetch(`${process.env.AUTH_URL}/API/verify`,
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${token}`,
      },
    );
    if (!response.ok) {
      return res.status(403).json('Unauthorized');
    } else {
      next();
    }
  } catch (err) {
    return res.status(500).json('Auth service error');
  }
}

```

Alle ruter på Apien kører en middleware function som checker om der er en valid bruger session før den udføre requests

```

app.get('/user/getUser/:id', auth, async (req: any, res: any) => {
  let id: number = await parseInt(req.params.id);
  let result = await prisma.users.findUnique({
    where: { id: id },
  });
  if (result == null) {
    res.status(400).json('No user found');
  } else {
    res.status(200).json(result);
  }
});

```

Billedet viser et eksempel på en get rute der udstedes af apien. Der finder en unik bruger ud fra et id.

Begrænsninger

Tidspres:

Vores tid har været presset og derfor har det også været svært at opbygge en fuldendt app. Vi har et par løse ender der mangler det sidste, men er nødsaget til at ikke kunne fuldføre alt pga. manglende tid.

Hjemmeside:

Design af hjemmeside blev ikke nået og derfor bruger den pt samme design som mobil appen.

Sikkerhed:

Det hele kører igennem HTTP hvilket resultere i mangel på sikkerhed da data ikke bliver encrypted når den sendes over internettet.

Api'er:

De to api'er har minimal error handling og hvis der var mere ville der laves mere uddybende error handling og response til requests. Der mangler også funktionalitet til at få ugentlig score.